

Practical Session 17: Building Agentic Iteration Loops

Hands-On: A Root-Finding Loop and a Credit-Risk Stress Loop

Juan F. Imbet

Paris Dauphine – PSL University

June 26, 2026

- 1 Recap: What You Need
- 2 Guided Problems
- 3 Mini Case Study and Discussion
- 4 Solutions

Recap: The Loop and Its Three Building Blocks

The goal-directed loop

Observe → **Propose** → **Act** → **Evaluate** → **Revise**, until $\text{ok}(s)$ holds or a stopping rule fires. Genuine iteration: a_{t+1} depends on feedback e_{t+1} .

- **Agents** — *who* acts (worker, critic, validator, planner, refactorer). Value = the durable change to state.
- **Skills** — *how* recurring work is done the same way twice (a skill $\approx$ a function).
- **Hooks** — *what* the work may never violate; fire automatically, outside the agent's control.

The rule we will exploit twice today

Never let the agent's own "done" end the loop. Termination must hang on a check the agent does *not* control — a **tolerance hook**.

Recap: Formulas and Rules for Today's Problems

Root finding (sign-and-magnitude search): from a bracket $[a, b]$ with $f(a) < 0 < f(b)$, the root lies between; weight the next guess toward the endpoint with the *smaller* $|f|$ (false-position intuition). Success: $|f(x)| < 10^{-3}$.

Expected loss:

$$\text{EL} = \text{PD} \times \text{LGD} \times \text{EAD}, \quad \text{EL (bps of EAD)} = \text{PD} \times \text{LGD} \times 10^4.$$

Illustrative bank credit policy (today's success criterion):

- Stressed EL ≤ 150 bps $\Rightarrow$ *lend* (clean).
- $150 < \text{stressed EL} \leq 500$ bps $\Rightarrow$ *conditional / modify*.
- Stressed EL > 500 bps $\Rightarrow$ *decline*.
- “Low risk” permitted *only if* base ≤ 50 bps *and* stressed ≤ 150 bps.

All figures today are illustrative and internally consistent (EL = PD · LGD · EAD in every column).

How to Work the Next Three Problems

- ❶ **P1 — IRR loop (transfer):** apply Example 1's sign-and-magnitude search to solve for an IRR from feedback alone.
- ❷ **P2 — The success gate:** write the tolerance hook in bash that vetoes a premature "success."
- ❸ **P3 — Credit mini case:** compute base vs. stressed EL and make the lend / modify / decline call.

THE BIGGER PICTURE

P1 and P2 are the *same loop* from two angles: P1 is the agent's directed search; P2 is the guardrail that defines "done." P3 scales the idea up to a judgment a committee must accept.

Companion notebooks: `code/notebooks/17-loops-goals-iterations/` — run the evaluator, the stop hook, and the EL reconciliation end-to-end.

P1 — Apply the Loop to an IRR (Transfer Task)

This is Example 1's loop with x renamed r and f renamed NPV — the same machine, only the residual changes.

Project cash flows: -100 at $t = 0$, $+60$ at $t = 1$, $+60$ at $t = 2$. Find the rate r (the IRR) with

$$\text{NPV}(r) = -100 + \frac{60}{1+r} + \frac{60}{(1+r)^2} = 0, \quad \text{tolerance } |\text{NPV}| < 0.05.$$

Derivatives are **forbidden** — you may only evaluate NPV at a trial rate and read the result. (Sign rule: $\text{NPV} > 0 \Rightarrow$ the rate is too *low*, so raise r .)

The loop has already run two iterations:

r	$\text{NPV}(r)$	Status
0.00	+20.00	Positive — rate too low
0.20	−8.33	Negative — overshoot

Your tasks:

- 1 Confirm the first two evaluations *bracket* the root. Where does the IRR lie?
- 2 From the *sign and magnitude* of NPV, propose the next two trial rates r_3, r_4 .
- 3 Are you within tolerance yet? Which loop component decides — the agent or the hook?

P2 — Write the Tolerance Success-Gate Hook *from spec*

Do not copy code — build it from the specification. The IRR evaluator of P1 appends one line per call to `run-log.txt`, the last line being the current state on disk.

Specification

Write a bash stop hook that:

- reads the *latest* logged NPV from `run-log.txt`;
- exits 0 (allow “success”) *only if* $|\text{NPV}| < 0.05$;
- otherwise prints a clear, human-readable message and exits 1, which **blocks** the agent from declaring success.

Why this matters

This single script is the difference between *feedback-driven iteration* and *premature success*. Without it, the agent can announce “done” at $r = 0.135$ ($|\text{NPV}| = 0.56$) and be believed.

Plan first: how do you grab the last line’s number (`tail/grep/cut`), and how do you test an absolute-value inequality in shell? Then write it. The reference solution follows later.

P3 — Credit Mini Case: Atlas Foods

Atlas Foods (fictional; figures illustrative) seeks a committed revolver. **EAD = \$20 m.**
The base-case PD model and LGD agent report:

Input	Base case	Stressed (adverse)
Probability of default (PD)	2.0%	8.0%
Loss given default (LGD)	35%	50%
Exposure at default (EAD)	\$20 m	\$20 m

The stressed PD and LGD are the loop's outputs after applying a combined shock (revenue -20% , margin -300 bps, rate $+250$ bps, DSO $+25$ days), with recovery deteriorating on illiquid collateral.

Your tasks:

- 1 Compute base and stressed EL, in *dollars* and in *bps of EAD*.
- 2 Apply the policy (recap slide): lend / modify / decline?
- 3 May the memo say "low risk"? Which *hook* decides, and why is that better than leaving it to the writer?

Mini Case Study: Designing the Atlas Loop

You are asked to wrap the Atlas analysis in a *goal-directed loop* rather than a one-shot memo. Sketch, in two minutes with a neighbour:

- **Goal file** — state the success criterion, a stopping rule, and one anticipated failure mode.
- **Agents** — which two or three roles are genuinely needed (resist excessive delegation)?
- **Hooks** — which automatic checks must fire, and *when* (pre- vs. post-action)?
- **Escalation trigger** — what condition forces a human into the loop before it proceeds?
- **Pattern selection** — which of the lecture's *five* design patterns best fits the Atlas credit workflow? Name it and justify it.

THE BIGGER PICTURE

There is no single right answer — but a good design names a *checkable* goal, a *guaranteed-terminating* stopping rule, at least one *outcome-tied* hook, and an escalation point that does not depend on the agent's goodwill.

Discussion Prompt

Debate (5 minutes)

“The Atlas loop converged: every hook is green and the skeptic signed off. Should the bank lend?”

Argue both sides, using the lecture’s vocabulary:

- **Goodhart / fake convergence:** could the hooks be green while the true goal is unmet? What would gaming look like here?
- **Accountability:** the skeptic’s sign-off is *not* sign-off. Who owns the decision, and what does the unsigned block add?
- **Audit trail:** if a regulator asks “where did the stressed 8% PD come from?” three months later, can you answer?

The point

A green loop produces a *defensible recommendation*, not a decision. “Lend” is a human act that the loop can evidence but never authorize.

P1 Solution — The IRR Bracket Tightens

(a) $\text{NPV}(0.00) = +20.00 > 0$ and $\text{NPV}(0.20) = -8.33 < 0$: NPV goes positive→negative, so it crosses zero in between — the IRR lies in $(0.00, 0.20)$. (b) Linear interpolation across the bracket, $0 + \frac{20}{20+8.33} \times 0.20 \approx 0.14$, points near 14%; test there and tighten on each sign change:

r	$\text{NPV}(r)$	Reasoning
0.00	+20.00	Positive; rate too low.
0.20	−8.33	Negative; bracket $(0.00, 0.20)$.
0.13	+0.09	Slightly positive; root in $(0.13, 0.20)$.
0.135	−0.56	Negative; root in $(0.13, 0.135)$.
0.131	−0.04	$ \text{NPV} < 0.05$: $\text{IRR} \approx 13.1\%$.

The same machine. This is literally Example 1’s sign-and-magnitude search with $x \rightarrow r$ and $f \rightarrow \text{NPV}$: bracket on a sign change, then weight the next guess toward the endpoint with the smaller $|\text{NPV}|$.

Who decides “done”?

The hook, not the agent. At $r = 0.135$, $|\text{NPV}| = 0.56$ — still above tolerance. The agent proposes; the tolerance hook (P2) is the only thing allowed to certify convergence.

P2 Solution — The Success-Gate Hook

```
#!/usr/bin/env bash
# stop-hook: veto "success" unless the latest |NPV| < 0.05
set -euo pipefail

last=$(grep -o 'NPV=[-0-9.eE]*' run-log.txt | tail -n1 | cut -d= -f2)
ok=$(python -c "print(1 if abs(float('$last')) < 0.05 else 0)")

if [ "$ok" -ne 1 ]; then
    echo "BLOCKED: |NPV|=$last >= 0.05 -- not converged" >&2
    exit 1
    # nonzero exit = veto: the loop may NOT stop
fi
echo "OK: |NPV| < 0.05 -- converged"
exit 0
```

Key properties: it reads the *last* line (the current state on disk), it is *deterministic* (same log → same verdict), and the agent cannot skip it — the harness treats the nonzero exit as a hard veto. This is the seven-step template's step 5 (validate) made real.

P3 Solution — Atlas Foods Expected Loss

Base case:

$$EL = 0.020 \times 0.35 \times \$20\text{m} = \$0.14\text{m} = 70 \text{ bps of EAD.}$$

Stressed (adverse):

$$EL = 0.080 \times 0.50 \times \$20\text{m} = \$0.80\text{m} = 400 \text{ bps of EAD.}$$

Decision

Stressed EL = 400 bps falls in $(150, 500]$ $\Rightarrow$ **conditional / modify** — not a clean lend, not a decline. Reasonable conditions: a smaller facility, tighter covenants, more collateral (to cut LGD), and pricing that compensates for the stressed EL.

May the memo say “low risk”? No. The “*low risk*” gate requires base ≤ 50 and stressed ≤ 150 bps; here base = 70 and stressed = 400. **The hook decides, not the writer** — because language is a control surface, and the phrase must be earned by the arithmetic.

Mini-Case Solution — Which Design Pattern?

Intended answer

The **credit-model: build → stress-test → red-team → finalize** pattern — a specialization of *scaffold-draft-audit-repair* with an explicit adversarial skeptic stage.

Why this one, not the others:

- The goal is a *judgment* (an acceptable committee memo), not a single machine-checkable number, so *generate-test-revise* alone is too thin.
- It is *high-stakes*: the two stages that distinguish this pattern — *mandatory stress testing* (adverse and severe, not just base) and a *red-team* skeptic that demands sensitivities and peer benchmarks — are exactly what Atlas needs.
- It *finalizes* into a governed artifact with an unsigned sign-off block, encoding SR 11-7 governance. Sub-patterns still nest inside (the PD-model build stage is *generate-test-revise*; the red-team is *planner-executor-critic*).

What you practised:

- Directed search from *feedback alone* (P1) — the engine inside every loop in the book.
- A *tolerance hook* (P2) that makes “done” external to the agent.
- EL reconciliation and a *policy-gated* lend decision (P3) — the credit loop in miniature.

THE BIGGER PICTURE

The root-finder and the credit memo are the *same machine*: a state variable, a validator, and a definition of “residual.” Only the residual changes — a number, a failing test, or a committee’s objection.

Take it further: `code/notebooks/17-loops-goals-iterations/` runs both loops end-to-end. See book Chapter 17 and Appendices A–F for the full treatment and the concrete tooling.